Atividade: Reconhecimento de Pokémon em Vídeo com Deep Learning

Machine Learning e Deep Learning · 6º período de Engenharia de Software · Nov 3, 2026 · @Thais

Contexto

Na atividade anterior, uma CNN com transfer learning (MobileNetV2) reconheceu gestos de pedra, papel e tesoura pela webcam, uma foto por vez. Agora o sistema passa a trabalhar com **vídeo**.

Este documento traz um **exemplo completo e funcionando**: um modelo que reconhece 3 objetos de mesa (caneca, estojo e grampeador) em um vídeo. O trabalho do grupo é **adaptar esse exemplo** para identificar **pelo menos 3 Pokémon em um vídeo, vistos de vários ângulos**, e implementar três melhorias que o exemplo ainda não tem (Parte 4).

O que muda em relação à atividade anterior:

- **A fonte dos dados é vídeo, não foto.** Vocês gravam vídeos e um script extrai os quadros (frames) automaticamente.
- **O mesmo objeto muda muito de aparência.** Um objeto visto de costas pode não se parecer nada com ele de frente. O modelo precisa ver todos os ângulos no treino.
- **O risco de vazamento de dados é maior.** Frames vizinhos de um vídeo são quase idênticos, então a validação passa a ser separada **por vídeo inteiro**, e não por frame.

A atividade tem cinco partes: gravar e extrair os dados, treinar o modelo, reconhecer objetos em um vídeo, desenvolver o trabalho com Pokémon e responder às perguntas descritivas.

Requisitos: Python 3 e as bibliotecas `pip install tensorflow numpy matplotlib opencv-python pillow`. Todos os arquivos de código ficam na mesma pasta.

Parte 1: Gravação dos vídeos e extração de frames

Em vez de fotografar pela webcam, você vai **gravar vídeos curtos** (10 a 30 segundos) de cada objeto e deixar um script transformar os vídeos em fotos. Para testar o exemplo, use três objetos que estejam na sua mesa: uma caneca, um estojo e um grampeador (ou outros três que tiver à mão).

Como usar:

1. Crie a pasta `videos/` com uma subpasta por classe: `videos/caneca/`, `videos/estojo/`, `videos/grampeador/` e `videos/fundo/` (vídeos **sem** nenhum objeto: a mesa vazia, a parede, a mão sozinha).
2. Grave **vários vídeos por classe** (mínimo de 3). Em cada vídeo, gire o objeto ou ande em volta dele. Dê nomes que digam o que foi gravado, por exemplo `v01_frente_claro.mp4`, `v02_costas_escuro.mp4`.
3. Salve os dois códigos abaixo (`utils.py` e `extrair_frames.py`) na mesma pasta onde está `videos/`.
4. Execute `python extrair_frames.py`.

As fotos são salvas em `data/<classe>/`, com o nome do vídeo dentro do nome da foto (`v01_frente_claro__000120.jpg`). Isso é essencial: o `train.py` usa esse nome para separar treino e validação **por vídeo**.

O script aproveita 4 frames por segundo, no máximo 120 por vídeo, e descarta frames quase iguais ao anterior (quando a câmera fica parada). No final, avisa se alguma classe ficou com muito menos fotos que as outras.

Código: utils.py (funções compartilhadas; a imagem precisa ser preparada do mesmo jeito no treino e na detecção)

```
"""
utils.py - funcoes compartilhadas entre extracao, treino e deteccao.
REGRA DE OURO: a imagem deve ser preparada EXATAMENTE do mesmo jeito
no treino e na deteccao. Por isso esta funcao fica num arquivo so.
"""
import cv2
import numpy as np

IMG_SIZE = 224
VIDEO_EXT = (".mp4", ".mov", ".avi", ".mkv", ".webm")


def letterbox(frame, size=IMG_SIZE):
    """Redimensiona mantendo a proporcao e completa com bordas pretas.
    Assim o objeto nao fica 'esticado' nem cortado."""
    h, w = frame.shape[:2]
    scale = size / max(h, w)
    nh, nw = int(round(h * scale)), int(round(w * scale))
    resized = cv2.resize(frame, (nw, nh), interpolation=cv2.INTER_AREA)
    canvas = np.zeros((size, size, 3), dtype=np.uint8)
    y0, x0 = (size - nh) // 2, (size - nw) // 2
    canvas[y0:y0 + nh, x0:x0 + nw] = resized
    return canvas
```

Código: extrair_frames.py

```
"""
Passo 1 - Extrai frames dos videos gravados.
Uso: python extrair_frames.py

Estrutura esperada (uma subpasta por classe, varios videos em cada):
  videos/
    caneca/      v01_frente_claro.mp4  v02_costas_escuro.mp4 ...
    estojo/      ...
    grampeador/  ...
    fundo/       (videos SEM nenhum objeto: so a mesa, a parede, a mao...)

Saida: data/<classe>/<nome_do_video>__<numero_do_frame>.jpg
O nome do video fica no nome da foto. O train.py usa isso para separar
treino e validacao POR VIDEO (evita vazamento de dados).
"""
import os
import cv2
import numpy as np
from utils import letterbox, VIDEO_EXT

VIDEOS_DIR = "videos"
DATA_DIR = "data"
FRAMES_POR_SEGUNDO = 4      # quantos frames aproveitar por segundo de video
MAX_POR_VIDEO = 120         # limite por video (equilibra as classes)
DIFF_MINIMA = 6.0           # descarta frame quase igual ao ultimo salvo (0 =
desliga)

if not os.path.isdir(VIDEOS_DIR):
    raise SystemExit(f"Crie a pasta '{VIDEOS_DIR}/' com uma subpasta por
classe.")

classes = sorted(d for d in os.listdir(VIDEOS_DIR)
                 if os.path.isdir(os.path.join(VIDEOS_DIR, d)))
print("Classes encontradas:", classes)

resumo = {}
for cls in classes:
    os.makedirs(os.path.join(DATA_DIR, cls), exist_ok=True)
    videos = sorted(f for f in os.listdir(os.path.join(VIDEOS_DIR, cls))
                    if f.lower().endswith(VIDEO_EXT))
    total_cls = 0
    for vid in videos:
        nome = os.path.splitext(vid)[0].replace("__", "_")
        cap = cv2.VideoCapture(os.path.join(VIDEOS_DIR, cls, vid))
```

```
        fps = cap.get(cv2.CAP_PROP_FPS) or 30
        passo = max(1, int(round(fps / FRAMES_POR_SEGUNDO)))
        i, salvos, ultimo = 0, 0, None
        while salvos < MAX_POR_VIDEO:
            ok, frame = cap.read()
            if not ok:
                break
            if i % passo == 0:
                img = letterbox(frame)
                pequeno = cv2.resize(img, (32, 32)).astype(np.float32)
                if ultimo is None or np.abs(pequeno - ultimo).mean() >=
DIFF_MINIMA:
                    cv2.imwrite(os.path.join(DATA_DIR, cls, f"
{nome}__{i:06d}.jpg"), img)
                    salvos += 1
                    ultimo = pequeno
            i += 1
        cap.release()
        total_cls += salvos
        print(f"  {cls}/{vid}: {salvos} frames")
    resumo[cls] = total_cls

print("\nTotal por classe:", resumo)
menor, maior = min(resumo.values()), max(resumo.values())
if maior > 1.5 * max(menor, 1):
    print("ATENCAO: classes desbalanceadas. Grave mais videos da classe menor
"
          "ou reduza MAX_POR_VIDEO.")
print("Pronto! Agora rode: python train.py")
```

Parte 2: Treino do modelo

Como usar:

1. Salve o código abaixo como `train.py`, na mesma pasta de `utils.py` e `data/`.
2. Execute `python train.py`.
3. Ao final, o script salva o modelo (`modelo.keras`), a lista de classes (`class_names.json`), o gráfico `curvas.png` e o PDF da entrega (`dataset_imagens.pdf`). No terminal aparecem a matriz de confusão, a precisão e o recall de cada classe.

O treino usa transfer learning como na atividade anterior: a MobileNetV2 fica congelada e só a camada final nova aprende. A diferença é que a **validação é separada por vídeo**: 20% dos vídeos de cada classe vão inteiros para a validação, e o terminal mostra quais foram. O

`EarlyStopping` interrompe o treino quando a loss de validação para de melhorar e devolve os melhores pesos.

O PDF da entrega é gerado no final do script. Ele mostra até 300 miniaturas por classe (uma amostra espaçada) e informa quantas imagens e quantos vídeos cada classe tem.

Código: train.py

```
"""
Passo 2 - Treino com Transfer Learning (MobileNetV2).
Uso: python train.py

A base (MobileNetV2, pre-treinada na ImageNet) fica congelada e so a
"cabeca" nova aprende, igual a atividade anterior.
A diferenca: a validacao e separada POR VIDEO.
"""
import os
import json
import random
from collections import defaultdict
import numpy as np
import tensorflow as tf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

DATA_DIR = "data"
IMG_SIZE = 224
BATCH = 16
EPOCHS = 8
VAL_FRACTION = 0.2          # fracao de VIDEOS (nao de fotos) para validacao
SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)

class_names = sorted(d for d in os.listdir(DATA_DIR)
                     if os.path.isdir(os.path.join(DATA_DIR, d)))
print("Classes:", class_names)

# ---- Split POR VIDEO ------------------------------------------------------
# Frames do mesmo video sao quase iguais. Se um video tiver frames no treino
e
# na validacao, a acuracia fica enganosamente alta. Por isso cada video
inteiro
# vai OU para treino OU para validacao. O nome do video vem antes de "__".
train_paths, train_y, val_paths, val_y = [], [], [], []
for idx, cls in enumerate(class_names):
    por_video = defaultdict(list)
```

```
    for f in sorted(os.listdir(os.path.join(DATA_DIR, cls))):
        if f.lower().endswith(".jpg"):
            por_video[f.split("__")[0]].append(os.path.join(DATA_DIR, cls,
f))
    videos = sorted(por_video)
    if len(videos) < 2:
        raise SystemExit(f"'{cls}' tem {len(videos)} video(s). Grave pelo
menos 3 por classe.")
    random.Random(SEED).shuffle(videos)
    n_val = max(1, round(len(videos) * VAL_FRACTION))
    val_videos = set(videos[:n_val])
    for v in videos:
        destino = (val_paths, val_y) if v in val_videos else (train_paths,
train_y)
        destino[0].extend(por_video[v]); destino[1].extend([idx] *
len(por_video[v]))
    n_tr = sum(len(por_video[v]) for v in videos if v not in val_videos)
    n_va = sum(len(por_video[v]) for v in val_videos)
    print(f"  {cls}: {n_tr} treino / {n_va} validacao | videos de validacao:
{sorted(val_videos)}")


def load(path, label):
    img = tf.io.read_file(path)
    img = tf.image.decode_jpeg(img, channels=3)
    img = tf.image.resize(img, (IMG_SIZE, IMG_SIZE))
    return img, label


def make_ds(paths, labels, training):
    ds = tf.data.Dataset.from_tensor_slices((paths, labels))
    if training:
        ds = ds.shuffle(len(paths), seed=SEED)
    ds = ds.map(load, num_parallel_calls=tf.data.AUTOTUNE).batch(BATCH)
    return ds.prefetch(tf.data.AUTOTUNE)


train_ds = make_ds(train_paths, train_y, True)
val_ds = make_ds(val_paths, val_y, False)

# Data augmentation (so atua no treino). O flip horizontal simula o objeto
# virado para o outro lado; a rotacao e o zoom simulam a camera se mexendo.
augment = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.10),
    tf.keras.layers.RandomZoom(0.20),
```

```
    tf.keras.layers.RandomTranslation(0.10, 0.10),
    tf.keras.layers.RandomBrightness(0.25),
    tf.keras.layers.RandomContrast(0.25),
])

base = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3), include_top=False,
weights="imagenet")
base.trainable = False

inputs = tf.keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
x = augment(inputs)
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
x = base(x, training=False)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.3)(x)
outputs = tf.keras.layers.Dense(len(class_names), activation="softmax")(x)
model = tf.keras.Model(inputs, outputs)

parar = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=3,
                                         restore_best_weights=True)

model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.summary()
hist = model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS, callbacks=
[parar])

model.save("modelo.keras")
with open("class_names.json", "w") as f:
    json.dump(class_names, f)

# ---- Curvas de treino vs validacao ----------------------------------------
plt.figure(figsize=(10, 4))
for i, (m, t) in enumerate([("accuracy", "Acuracia"), ("loss", "Loss")]):
    plt.subplot(1, 2, i + 1)
    plt.plot(hist.history[m], label="treino")
    plt.plot(hist.history["val_" + m], label="validacao")
    plt.title(t); plt.xlabel("epoca"); plt.legend()
plt.tight_layout(); plt.savefig("curvas.png")

# ---- Matriz de confusao + precisao/recall por classe ----------------------
y_true = np.array(val_y)
y_pred = np.argmax(model.predict(val_ds, verbose=0), axis=1)
k = len(class_names)
cm = np.zeros((k, k), dtype=int)
```

```
for t, p in zip(y_true, y_pred):
    cm[t, p] += 1
print("\nMatriz de confusao (linhas = real, colunas = previsto):")
print(" " * 11 + " ".join(f"{c[:8]:>8}" for c in class_names))
for i, c in enumerate(class_names):
    print(f"{c[:10]:>10} " + " ".join(f"{v:>8}" for v in cm[i]))
print("\nClasse        precisao   recall")
for i, c in enumerate(class_names):
    prec = cm[i, i] / max(cm[:, i].sum(), 1)
    rec = cm[i, i] / max(cm[i, :].sum(), 1)
    print(f"{c[:12]:<12} {prec:>9.2%} {rec:>8.2%}")
print(f"\nAcuracia de validacao: {np.trace(cm) / cm.sum():.2%}")


# ---- PDF do dataset (entrega obrigatoria) ---------------------------------
from PIL import Image, ImageDraw, ImageFont


def gerar_pdf_dataset(data_dir="data", saida="dataset_imagens.pdf", cols=10,
                      qualidade=70, max_por_classe=300):
    """Uma secao por classe. Se houver muitas fotos, pega uma amostra
espacada
    (max_por_classe) para o PDF nao ficar gigante. A legenda mostra quantos
    videos geraram as fotos daquela classe."""
    pag_w, pag_h, margem, topo = 1240, 1754, 40, 110
    cell = (pag_w - 2 * margem) // cols
    rows = (pag_h - topo - margem) // cell
    por_pagina = cols * rows
    try:
        fonte = ImageFont.load_default(size=28)
    except TypeError:
        fonte = ImageFont.load_default()
    paginas = []
    classes = sorted(d for d in os.listdir(data_dir)
                     if os.path.isdir(os.path.join(data_dir, d)))
    for classe in classes:
        pasta = os.path.join(data_dir, classe)
        fotos = sorted(f for f in os.listdir(pasta) if
f.lower().endswith(".jpg"))
        n_videos = len({f.split("__")[0] for f in fotos})
        total = len(fotos)
        if total > max_por_classe:
            idx = np.linspace(0, total - 1, max_por_classe).astype(int)
            fotos = [fotos[i] for i in idx]
        n_pag = max(1, -(-len(fotos) // por_pagina))
        for p in range(n_pag):
```

```
            pagina = Image.new("RGB", (pag_w, pag_h), "white")
            draw = ImageDraw.Draw(pagina)
            draw.text((margem, 40),
                      f"Classe: {classe} | {total} imagens de {n_videos}
videos | pagina {p + 1}/{n_pag}",
                      fill="black", font=fonte)
            for i, nome in enumerate(fotos[p * por_pagina:(p + 1) *
por_pagina]):
                img = Image.open(os.path.join(pasta, nome)).convert("RGB")
                img.thumbnail((cell - 6, cell - 6))
                pagina.paste(img, (margem + (i % cols) * cell + 3, topo + (i
// cols) * cell + 3))
            paginas.append(pagina)
    paginas[0].save(saida, save_all=True, append_images=paginas[1:],
                    quality=qualidade, resolution=150)
    print(f"PDF salvo: {saida} ({len(paginas)} paginas)")


gerar_pdf_dataset(DATA_DIR)
print("\nPronto! Modelo salvo em modelo.keras. Agora rode: python detectar.py
meu_video.mp4")
```

Parte 3: Reconhecer objetos em um vídeo

Como usar:

1. Salve o código abaixo como `detectar.py`, na mesma pasta de `utils.py`, `modelo.keras` e `class_names.json`.
2. Para analisar um vídeo: `python detectar.py meu_video.mp4`. O script gera `saida_meu_video.mp4`, com o nome do objeto e as barras de probabilidade escritos em cada frame.
3. Para testar ao vivo: `python detectar.py 0` (webcam). Aperte `q` para sair.
4. Para processar sem abrir janela (mais rápido): `MOSTRAR=0 python detectar.py meu_video.mp4` (no Windows: `set MOSTRAR=0` antes do comando).

Neste exemplo, **cada frame é classificado sozinho**: o rótulo mostrado é sempre a classe de maior probabilidade naquele frame, mesmo que o modelo esteja em dúvida. Rode em um vídeo seu e observe: o nome "pisca" entre classes quando o objeto gira ou a câmera treme. Corrigir isso faz parte do trabalho (Parte 4).

Código: detectar.py

```
"""
Passo 3 - Reconhece objetos em um video (ou na webcam).
Uso:
  python detectar.py meu_video.mp4     -> gera saida_meu_video.mp4
  python detectar.py 0                 -> usa a webcam (q = sair)

Cada frame e classificado sozinho: o modelo olha o frame, devolve as
probabilidades e o rotulo de maior probabilidade e escrito na tela.
A preparacao da imagem e a mesma do treino (letterbox do utils.py).
"""
import sys
import os
import json
import cv2
import numpy as np
import tensorflow as tf
from utils import letterbox

CLASSE_VAZIA = "fundo" # classe que significa "nenhum objeto"

model = tf.keras.models.load_model("modelo.keras")
class_names = json.load(open("class_names.json"))

fonte = sys.argv[1] if len(sys.argv) > 1 else "0"
webcam = fonte.isdigit()
cap = cv2.VideoCapture(int(fonte) if webcam else fonte)
if not cap.isOpened():
    raise SystemExit(f"Nao consegui abrir: {fonte}")
fps = cap.get(cv2.CAP_PROP_FPS) or 30

writer = None
if not webcam:
    w, h = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),
int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    saida = f"saida_{os.path.splitext(os.path.basename(fonte))[0]}.mp4"
    writer = cv2.VideoWriter(saida, cv2.VideoWriter_fourcc(*"mp4v"), fps, (w,
h))

n = 0
while True:
    ok, frame = cap.read()
    if not ok:
        break
    if webcam:
        frame = cv2.flip(frame, 1)
    img = cv2.cvtColor(letterbox(frame), cv2.COLOR_BGR2RGB).astype("float32")
```

```
    probs = model.predict(img[None], verbose=0)[0]
    i = int(np.argmax(probs))
    rotulo = class_names[i]

    # --- desenho ---
    texto = "nenhum objeto" if rotulo == CLASSE_VAZIA else rotulo
    cor = (0, 200, 0) if rotulo != CLASSE_VAZIA else (0, 200, 255)
    cv2.rectangle(frame, (0, 0), (frame.shape[1], 40), (0, 0, 0), -1)
    cv2.putText(frame, f"{texto} ({probs[i]:.0%})", (10, 28),
                cv2.FONT_HERSHEY_SIMPLEX, 0.9, cor, 2)
    for j, c in enumerate(class_names):
        y = 55 + j * 22
        cv2.rectangle(frame, (10, y), (10 + int(probs[j] * 150), y + 14), (0,
200, 255), -1)
        cv2.putText(frame, c, (170, y + 13), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
(255, 255, 255), 1)
    cv2.putText(frame, f"t = {n / fps:5.1f}s", (10, frame.shape[0] - 15),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)

    if writer is not None:
        writer.write(frame)
    if webcam or os.environ.get("MOSTRAR", "1") == "1":
        cv2.imshow("Reconhecimento com CNN", frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    n += 1

cap.release()
if writer is not None:
    writer.release()
    print(f"Video anotado salvo em: {saida}")
cv2.destroyAllWindows()
```

Parte 4: Trabalho com Pokémon

Agora é a vez do grupo. O trabalho pode ser feito em trios: adaptem o exemplo para **identificar pelo menos 3 Pokémon em um vídeo, vistos de vários ângulos**, com dados gravados por vocês, e implementem as três melhorias abaixo.

Adaptação dos dados:

1. Escolham pelo menos 3 Pokémon e consigam um objeto físico de cada (pelúcia, boneco, carta, impressão). Se possível, escolham dois parecidos na cor (por exemplo, Pikachu e Psyduck, ambos amarelos): isso deixa o problema mais interessante.

2. Troquem as pastas de `videos/` pelos nomes dos seus Pokémon. Mantenham a classe `fundo`.
3. Para cada Pokémon, gravem **no mínimo 4 vídeos**, cobrindo os ângulos da tabela abaixo, e no mínimo 4 vídeos da classe `fundo`.
4. Gravem cada classe em **dois cenários**: fundo claro e fundo escuro, divididos entre os vídeos (por exemplo, 2 vídeos em cada).

Ângulo	Como gravar
Frente	Câmera na altura do Pokémon, de frente
Costas	Gire o Pokémon 180°
Lados (esquerdo e direito)	Gire 90° para cada lado
De cima	Câmera inclinada para baixo, cerca de 45°
De baixo	Câmera abaixo do Pokémon, apontando para cima
Perto e longe	Pokémon ocupando quase o quadro todo e depois só um terço dele

Melhorias que o grupo deve implementar:

1. **Suavização e limiar de confiança (`detectar.py`).** Em vez de usar só as probabilidades do frame atual, usem a média das probabilidades dos últimos N frames. Se a maior probabilidade média ficar abaixo de um limiar (por exemplo, 60%), o sistema deve mostrar `?` em vez de um nome.
2. **Linha do tempo (`detectar.py`).** Ao final do vídeo, o script deve imprimir em que trechos cada Pokémon apareceu e a lista dos Pokémon encontrados, ignorando trechos muito curtos (menos de meio segundo). Exemplo de saída esperada:

```
Linha do tempo (Pokemon identificados):
     0.0s a    6.4s  ->  pikachu
     8.1s a   15.0s  ->  charmander
    16.2s a   24.8s  ->  bulbasaur
Pokemon encontrados no video: ['bulbasaur', 'charmander', 'pikachu']
```

3. **Fine-tuning (`train.py`).** Depois do treino atual (base congelada), acrescentem uma segunda fase que descongela as últimas camadas da MobileNetV2 e continua o treino com uma taxa de aprendizado bem menor. O gráfico `curvas.png` deve mostrar as duas fases e marcar onde começa o fine-tuning.

Vídeo de teste: gravem um vídeo de 30 a 60 segundos que **nunca foi usado no treino**, em que os 3 Pokémon aparecem um de cada vez, em ângulos variados e em um lugar diferente dos vídeos de treino. Rodem o `detectar.py` nele.

Cuidados com os dados:

- Mantenham um número parecido de fotos em todas as classes. O `extrair_frames.py` avisa quando há desequilíbrio.
- Cada vídeo deve misturar ângulos, em vez de um vídeo só de frente e outro só de costas. Como a validação é feita por vídeo, um ângulo que só aparece em um vídeo pode ficar de fora do treino.

Regra de entrega (só serão aceitos trabalhos com os itens 1 e 2):

1. `dataset_imagens.pdf`, gerado pelo `train.py`, com as imagens separadas por classe (os 3 ou mais Pokémon e a classe `fundo`).
2. O vídeo de teste anotado (`saida_<nome>.mp4`) e o print da linha do tempo impressa no terminal.
3. Os códigos adaptados (`train.py` e `detectar.py`), com comentários explicando as partes que o grupo implementou.
4. O gráfico `curvas.png` e a matriz de confusão (print do terminal).
5. Um relatório em PDF com as respostas da Parte 5.

Dicas

Sobre os dados

1. **Vídeo gera muitas fotos, mas poucas fotos diferentes.** 30 segundos a 30 fps são 900 frames quase iguais. Por isso o script pega só 4 por segundo e descarta repetidos. Mais vídeos variados valem mais que um vídeo longo.
2. **Separe por vídeo, nunca por frame.** Se o frame 100 está no treino e o frame 101 na validação, o modelo "acerta" porque já viu quase a mesma imagem. Acurácia de validação perto de 100% logo na primeira época é sinal de vazamento.
3. **O nome do arquivo é metadado.** Guardar o nome do vídeo dentro do nome da foto permite separar por vídeo, rastrear erros e saber de onde veio cada imagem. Usem nomes que descrevam a gravação (`v03_costas_escuro`).
4. **A classe `fundo` precisa ser variada.** Gravem a mesa vazia, a mão sem Pokémon, outros objetos. Se o fundo for sempre o mesmo, o modelo aprende "essa mesa = fundo".
5. **Cuidado com atalhos.** Se todo vídeo do Pikachu foi gravado na mesa branca e todo vídeo do Charmander na mesa de madeira, o modelo pode aprender a mesa, e não o Pokémon. Gravem todos os Pokémon em todos os cenários.

Pistas para as melhorias

1. **Suavização:** a classe `deque` do módulo `collections` guarda só os últimos N itens: `deque(maxlen=8)`. A cada frame, guardem o vetor `probs` nela e calculem `np.mean(..., axis=0)`. Testem janelas de 1, 8 e 15 frames e vejam o efeito.
2. **Limiar:** depois da média, comparem a maior probabilidade com o limiar antes de decidir o rótulo. Ajustem o valor olhando o vídeo anotado: alto demais e o sistema quase sempre diz `?`; baixo demais e ele volta a chutar.
3. **Linha do tempo:** guardem o rótulo de cada frame em uma lista. No final, percorram a lista agrupando frames seguidos com o mesmo rótulo. O tempo de um frame é `indice / fps`. Descartem trechos de `?`, de `fundo` e os muito curtos.
4. **Fine-tuning:** depois do primeiro `model.fit`, façam `base.trainable = True` e congelem de novo todas as camadas menos as últimas (por exemplo, 30). Depois de mudar o que é treinável, é **obrigatório** chamar `model.compile` de novo, agora com uma taxa de aprendizado em torno de 1e-5. Para o gráfico, juntem os dois históricos (`hist1.history` e `hist2.history`).

Sobre o treino

1. **Treinem a cabeça antes do fine-tuning.** Se descongelarem a base com a cabeça ainda aleatória, os gradientes grandes destroem os filtros pré-treinados.
2. **Fine-tuning pede taxa de aprendizado baixa.** Com 1e-3 no fine-tuning, a loss de validação costuma subir; com 1e-5, os pesos mudam aos poucos.
3. **Leiam as curvas.** Treino subindo e validação parada ou caindo é overfitting.
4. **Olhem a matriz de confusão, não só a acurácia.** Uma acurácia de 85% pode esconder um Pokémon que nunca é reconhecido. Vejam quais pares se confundem e pensem no motivo.
5. **Sem GPU?** Usem o Google Colab (Ambiente de execução > Alterar tipo > GPU). Façam upload da pasta `data/` compactada e rodem o `train.py` lá. O `detectar.py` roda bem na CPU.

Sobre o vídeo

1. **Mesma preparação no treino e na detecção.** O `letterbox` está em `utils.py` justamente para isso. Se o treino usar a imagem inteira e a detecção cortar o centro, o modelo vê imagens diferentes das que aprendeu.
2. **Classificação não é detecção.** Este modelo responde "qual Pokémon está nesta imagem", mas não "onde ele está". Com dois Pokémon no mesmo quadro, ele escolhe um só. Para localizar vários objetos, usa-se um modelo de detecção (como o YOLO), que desenha uma caixa em volta de cada um.

Parte 5: Perguntas

Responda com suas próprias palavras, de forma descritiva, usando exemplos do que você observou na prática. Cada pergunta pede um experimento: descreva o que foi feito, mostre o resultado (print, número ou trecho do vídeo) e explique.

1. Por que separar treino e validação por vídeo? Treine uma vez com o split por vídeo (o `train.py` original) e outra vez com um split aleatório por foto (sorteie 20% das fotos de todos os vídeos). Compare a acurácia de validação dos dois e o desempenho no vídeo de teste. Qual número representa melhor o desempenho real? Explique usando o conceito de vazamento de dados.

2. O fine-tuning ajudou? Compare o modelo ao final da fase 1 (só a cabeça) com o modelo após o fine-tuning, usando as curvas e a matriz de confusão. Depois, repita o fine-tuning com taxa de aprendizado 1e-3 em vez de 1e-5. O que aconteceu com a loss de validação? Por que a taxa precisa ser tão baixa?

3. O modelo reconhece o Pokémon de qualquer ângulo? Retreine o modelo deixando de fora todos os frames de costas de um dos Pokémon. Rode o `detectar.py` em um vídeo desse Pokémon de costas. O que o modelo previu? Que características ele parece usar para reconhecer cada Pokémon (cor, formato, orelhas, cauda)? Proponha duas estratégias para tornar o modelo mais robusto a ângulos novos.

4. O que acontece com um Pokémon que o modelo não conhece? Mostre para o `detectar.py` um quarto Pokémon que não está no treino. O modelo disse `?` ou afirmou ser um dos três? Com qual confiança? Explique por que a função softmax sempre distribui 100% entre as classes conhecidas e como o limiar e a classe `fundo` ajudam (ou não) nesse caso.

5. Classificação ou detecção? Grave um trecho com dois Pokémon no mesmo quadro e rode o `detectar.py`. Descreva o que aconteceu. Explique a diferença entre classificação de imagens (o que fizemos) e detecção de objetos, e descreva o que mudaria nos dados (anotação) e no modelo para que o sistema encontrasse e marcasse todos os Pokémon de um quadro ao mesmo tempo.